A 55-Addition Rank-23 Scheme for 3×3 Matrix Multiplication via Exact Two-Sided Minimization

Greg Sidebottom

Research note — logic repo (https://github.com/gsidebottom/logic), matmul track, 2026-07-05. The focused write-up of the 55-addition result and how to reproduce it; the earlier 56-era exploration (orbit-CSE search, database-wide side-floor census, SAT lower bounds) lives in the git history. Companion to `doc/matmul_53_3x3_schemes.md`*; every claim has a mechanical check (§7).*

Abstract

We multiply two 3×3 matrices with 23 multiplications and 55 additions — one below the previous record of 56 (Yinqi Sun, arXiv:2604.27645, Apr 2026), and the first sub-56 scheme for exact non-commutative 3×3 multiplication without change of basis. The algorithm is short and concrete: form 13 signed combinations of the 9 entries of A and 14 of B, take 23 products (each one A-combination times one B-combination), and recombine them with 28 further additions into the 9 entries of C — negation is free. It is a runnable program (`src/mm55.rs`; also `matmul/external/i19-55adds-slp.txt`) checked against the naive 27-multiplication triple loop on 500,000 random integer inputs **and** 50,000 non-commutative 2×2-block inputs — so it holds over any ring and recurses on block matrices — with an operation counter confirming exactly $23\times$ and $55\pm$ (`cargo test mm55`). Its correctness is additionally machine-checked in Lean 4 (`matmul/mm55proof/`): a sorry-free proof that, over a general non-commutative ring, the program's 9 outputs are the entries of $A \cdot B$ — and, in Mathlib's own `Matrix` API, that the scheme equals `A * B`.

The 56 record was the end of a rapid chain — 60 (Stapleton) $\to$ 59 (Mårtensson–Stankovski Wagner–Stapleton, MWS) $\to$ 58 $\times 3$ (Perminov) $\to$ 56 — every step driven by greedy common-subexpression elimination (CSE) on the linear forms. A scheme's additive cost splits into two *input sides* (the left/right linear factors of the 23 products) and one *output side* (the 9 outputs recombined from the products); greedy pair-extraction CSE returns only an *upper bound* per side. We minimize both sides **exactly**:

1. **Input sides** (`matmul/sidemin.py`): the minimum $\pm$-additions such that one addition chain over the 9 input entries covers every distinct factor row up to sign — Sun's chain-covering structure — by iterative deepening over helper values. It reproduces Sun's own 13+13 optima and both of his per-side impossibility certificates.
2. **Output side** — the new ingredient (`matmul/tcmin.py`): the **transposition principle** (Tellegen). For any addition-only linear map, $A(W) = A(W^{\mathsf{T}}) + (\text{inputs} - \text{outputs})$; the 9×23 output map W therefore costs $A(W^{\mathsf{T}}) + 14$, and W^{T} is a 9-input ternary map — *exactly the regime the input-side minimizer solves exactly*, and small ($A(W^{\mathsf{T}}) \approx 16$). So the output side is minimized **exactly and constructively** (transpose the chain), where greedy CSE — the engine of the entire 60 $\to$ 56 chain — fell short by an addition.

Combining the two on Perminov's `cn122` scheme (class `i19w225c4efh`, published at 58 additions, a de Groote class distinct from Sun's) gives exact output side 28 (greedy: 29) with input sides 13+14, i.e.

$$\mathbf{55 = 13 + 14 + 28},$$

the first sub-56 scheme. It is verified exactly (Brent equations over $\mathbb{Z}$) and functionally by an independent from-scratch checker — 5,000 random integer 3×3 products **and** 300 non-commutative 2×2-block products, operation count exactly 55. The same exact method independently confirms **Sun's own representative is output-optimal** (56 exact): 56 was optimal for his class, and 55 lives on a different class that had never been exactly output-minimized.

1 Notation and terminology

- **Sides.** A scheme's additive cost splits into two *input sides* — the A- and B-side forms, over the 9 entries of A and of B — plus one *output side*, the C-side forms computing the 9 outputs from the 23 products. Both are minimized exactly here: input sides by chain covering (§3), the output side by transposition (§4).
- **de Groote group, class, orbit, representative.** The de Groote symmetry group acts on schemes (sandwiching the three tensors by invertible matrices, plus the S_3 slot symmetries); two schemes are *equivalent* if one is the image of the other. The **orbit** of a scheme is its set of images under the group — identical to its equivalence **class**; a **representative** is any member. The additive cost varies across a class, but never the class itself.
- **Scheme identifiers** (`i12w219c23ci-008`, `i2w201c26fi-000`, …) are the file names of the stored schemes in the public HKS database, used here verbatim as opaque labels; the trailing `-N` indexes files sharing a prefix, and the prefix fields encode HKS's own cataloguing invariants (see the HKS database reference). Nothing relies on their semantics: "class X" always means the de Groote equivalence class of the scheme stored under that name, and every class identity we assert is established by our own exact fingerprint + witness machinery, not by name.
- **sign-SAT** (companion paper, §3.5): the Boolean satisfiability instance whose variables are the signs of the mod-2-supported coefficients. A covering term's sign is the XOR of its three factor signs, and each integer Brent equation with k covering terms and right-hand side δ becomes the cardinality constraint "exactly $(k-\delta)/2$ of the k terms are negative". Its solutions are exactly the valid ± 1 lifts of the mod-2 scheme (worked example in Appendix C).
- **$\mathbb{Z}$-verified**: checked exactly over the integers — all 729 Brent equations evaluated with the scheme's ± 1 coefficients in exact integer arithmetic, zero residuals. (Code output writes plain `Z` for $\mathbb{Z}$.)
- h^*: the star denotes the optimal value — h^* is the smallest helper count h at which the side-minimizer's search succeeds.
- **Closure normal form**: defined in §3 (Method); worked example in Appendix B.

2 Cost model and history

A bilinear scheme with r products computes $M_m = P_m \cdot Q_m$ from linear forms P_m (over the 9 entries of A), Q_m (over B), then each C entry as a linear form over the M_m. The **additive complexity** is the number of binary additions/subtractions in a straight-line program (SLP) computing all forms; unary negation is free; no change of basis (Karstadt–Schwartz accounting is *not* used). This is the model of Mårtensson–Wagner, Stapleton, Perminov, and Sun; Schwartz–Vaknin's 61 lives in the weaker with-basis-change model.

year	count	scheme / method	basis change
1976	98	Laderman, naive form	—
2023	61	Schwartz–Vaknin (pebbling + alt. basis)	**yes**
2024	62	Mårtensson–Wagner, Greedy-Potential on Laderman	no
2025 (Aug)	60	Stapleton (NN + Greedy-Potential); class `i2w201c26fi`	no
2025 (Dec)	59	MWS, arXiv:2601.05272; class `i19w203c23ci`	no
2025 (Dec)	58 (×3)	Perminov, arXiv:2512.21980; classes `i12w219c23ci`, `i46w213c23ci`, `i19w225c4efh`	no
2026 (Apr)	56	Sun, arXiv:2604.27645; cyclic image of `i46w213c23ci`	no
this note	**55 — record**	`i19w225c4efh`; exact input sides + transposition-exact output side	no

Every step from 62 to 56 used greedy CSE; ours is the first to minimize both sides exactly. Re-scored exactly (§5), the record chain also drops on its own representatives — the 59 scheme to 58, the three 58 schemes to 56/56/55, and three 60-addition classes of our own to 59 each. Among the record-chain classes the exact re-score puts **55 on one class** (`i19w225c4efh`) **and 56 on two** (Sun's `i46w213c23ci` and `i12w219c23ci`); a database-wide exact re-score (§8) is extending the survey.

3 The exact input-side minimizer (`matmul/sidemin.py`)

Problem. Given the 23 signed factor rows of one side (vectors in $\{-1, 0, +1\}^9$; rows equal to a basis vector or to ±another row are free), find the minimum number of steps of the form $v = \pm x \pm y$ (x, y earlier values, starting from the 9 basis vectors) such that every distinct multi-term row appears among the values up to global sign. This is Sun's chain-covering structure as a search problem: a single addition chain whose values include all 23 left (or right) factors, where greedy pair extraction can only build balanced trees of shared *pairs* and cannot reuse a sum of three, four, or five inputs.

Method. Iterative deepening on the number of *helpers* h (chain values that are not target rows); the optimum is (#distinct multi-term targets) $+ h^*$. The search uses what we call the **closure normal form**: because the pool of computed values only ever grows, covering a currently-derivable target can never hurt a later step and always costs exactly one addition — so the search covers every derivable target greedily until none remains (the *closure fixpoint*) and branches **only on which helper value to insert at a fixpoint**. Any optimal chain reorders into this form (an exchange argument: coverable-target steps commute forward past helper insertions), so restricting the search to it loses nothing. With one helper left, the helper must directly complete some uncovered target, shrinking the last level to an "enabling set". Helpers may be arbitrary integer vectors, doublings included — strictly more general than the pure/one-aux model of Sun's certificates, so an exhaustion at slack h here is the stronger impossibility statement.

Calibration against Sun's own certificates. His verifier proves each of his sides optimal by a reachability argument (no 12-addition chain covers U's 12 targets; no 12-addition chain with any single auxiliary covers V's 11). Our search independently reproduces all of it — U: 12 targets, $h^* = 1$, 13 additions (3 search nodes); V: 11 targets, $h^* = 2$, 13 additions (11 nodes); the $h = 0$ and $h = 1$ exhaustions match his two impossibility certificates — in 0.3 s total, every chain replay-verified. Micro-optima (5 hand-checkable cases) also pass.

4 The exact output-side minimizer (transposition principle)

The output side computes the 9 result entries $C_{pq} = \sum_m W_{pq,m} M_m$ from the 23 products: a linear map W, a 9×23 ternary matrix. Greedy pair-extraction CSE minimizes it only approximately (an upper bound). The exact minimum is the *shortest linear straight-line program* for W — NP-hard in general, and the reason the record chain never drove the output side to its true minimum.

The transposition principle (Tellegen; standard in the linear-circuit and signal-processing literature, and in algebraic complexity theory) resolves it. For an addition-only linear map — only ± of earlier values, no change of basis, negation free — the minimum additions of a map and its transpose differ by a fixed constant:

$$A(W) = A(W^{\mathsf{T}}) + \big(\text{inputs}(W) - \text{outputs}(W)\big).$$

(We verify the constant on hand-checkable cases: for M with $y_1 = x_1{+}x_2$, $y_2 = x_2{+}x_3$, $A(M) = 2$ and $A(M^{\mathsf{T}}) = 1 = 2 + (2 - 3)$; `tcmin.py`'s selftest checks four such.) For the output side W is 9×23 (23 inputs, 9 outputs), so $\mathbf{A(W) = A(W^{\mathsf{T}}) + 14}$, and W^{T} is a 23-output map over **9 inputs** — *exactly the shape the input-side minimizer solves exactly*, and small ($A(W^{\mathsf{T}}) \approx A(W) - 14 \approx 16$). Hence

$$\text{exact output side} = \texttt{sidemin}(\text{rows of } W^{\mathsf{T}} \text{ over 9 dims}) + 14,$$

computed in milliseconds. It is **constructive**: transposing the W^{T} addition chain by the standard adjoint (reverse the program; fan-out ↔ addition) yields an explicit $A(W^{\mathsf{T}})$+14-addition program for the 9 outputs, which `matmul/tcmin.py` emits and which we verify independently (Appendix A).

Impact. On Sun's own representative the exact output side is 30 — equal to his greedy count, so **Sun's scheme is output-optimal and 56 is optimal for his representative**. But on Perminov's `cn122` representative (class `i19w225c4efh`) the exact output side is **28**, where greedy CSE returns 29 — the missing addition greedy could not see. Combined with the exact input sides 13+14, that is a **55-addition scheme** (Appendix A). `tcmin.py`'s selftest pins both numbers (Sun 30, cn122 28) and the transposition constant.

5 Results

Exact totals per de Groote class (input sides by `sidemin`, output side by transposition; all schemes Brent-verified over $\mathbb{Z}$, class identities by exact fingerprint + witness machinery cross-checked against the HKS database; "published" = the count in the source paper):

de Groote class	publ.	exact (A+B+C)	representative
`i19w225c4efh` (Perminov cn122)	58	**55 = 13+14+28**	his published rep (m2)
`i46w213c23ci` (Sun / cn120)	56	56 = 13+13+30	Y. Sun's rep
		56 = 13+16+27	D. Perminov's cn120 rep
`i12w219c23ci` (Perminov cn119)	58	56 = 13+14+29	orbit rep (ours)
`i19w203c23ci` (MWS 59)	59	58 = 14+15+29	their published rep
`i106w191c347g` etc. (ours, 3)	—	59 = 15+15+29	orbit reps (ours)
`i2w201c26fi` (Stapleton 60)	60	60 = 16+16+28	orbit rep (ours)

- **55 on one class** (`i19w225c4efh`) — the record, verified in Appendix A — and provably *only* on this class in the slim tiers: for each of the 79,488 sides-$\leq$ 27 representatives (the complete orbits of all 84 floor-26/27 classes), replaying §4's transposition argument over GF(2) (`matmul/gf2min.py`) gives an exact GF(2) total that lower-bounds **every** integer sign model of that representative. The bound is ≥ 55 everywhere, and equals 55 only on 864 representatives of this one class.
- **56 on two classes** (`i46w213c23ci` = Sun's, and `i12w219c23ci`); before the exact re-score these read 56/58 under greedy. The exact output side beats greedy by one addition on the two Perminov classes (cn122 29 $\to$ 28, cn119 30 $\to$ 29) and by one on the cn120 representative (28 $\to$ 27).
- **No 54 exists anywhere in the published database — 55 is the additive minimum.** Two exhaustive tiers, both model-independent via the GF(2) bound:
  1. *Slim representatives* (GF(2) input sides ≤ 27, possible only in the 4 floor-26 + 80 floor-27 classes): all 79,488 were enumerated; GF(2) sides + exact GF(2) output side ≥ 55 on every one (and the 12-sign-model exact $\mathbb{Z}$ re-score independently bottoms out at 55).
  2. *Fat representatives* (input sides ≥ 28) of **every** class: the output-side cost depends only on the γ-support, whose de Groote orbit reduces to 784 canonical $(X \otimes Y)$-images per tensor slot (output-coordinate permutations are free, so X, Y range over S_3-cosets of GL$(3,2)$; validated against the full 56,448-image sweep). `matmul/cfloor.py` swept all 44,125 candidate slots of all 17,376 classes: **no image anywhere has XOR cover** ≤ 12, so the exact GF(2) output side is ≥ 27 on every representative of every class, and any fat representative totals $\geq 28 + 27 = 55$.

  Exactly 620 slots (616 classes, including the record class and Sun's) reach cover 13 — output side 27 — so those are the only places further (record-tying, never record-beating) 55s could live.

6 Contributions

- **A 55-addition rank-23 scheme** — the first below 56 — with an explicit, runnable, independently re-verified program (`src/mm55.rs`, `matmul/external/i19-55adds-slp.txt`).
- **Exact two-sided additive minimization** in this model: the input side as addition-chain covering (`sidemin.py`, calibrated against Sun's own optimality certificates), and the output side by the **transposition principle** (`tcmin.py`) — the ingredient the entire greedy-CSE record chain lacked. Both are exact and constructive.

- **An exact re-score of the record chain**, which improves every class on its own representative and fixes the true count per class (§5): 56 on two classes, 55 on one, and confirmation that Sun's representative was already output-optimal.
- **An additive-optimality theorem for the published catalogue**: no representative of any of the 17,376 published classes, under any sign model, multiplies 3×3 matrices with 23 products in fewer than 55 additions (§5) — so the record is not just current but final for this catalogue, and any future sub-55 scheme must come from a class outside it.

7 Reproduction

Prerequisites: `python3` (3.11+, no packages needed), `kissat` (sign-model enumeration; e.g. `brew install kissat`), a Rust toolchain (`cargo`) for the runnable scheme, and `elan` for the Lean proof. The database-wide steps additionally need the public scheme archive — rebuild it from the primary source with `python3 dbcheck.py fetch && python3 dbcheck.py convert` (downloads `schemes.tgz`, ∼43 MB, from the Linz algebra server and writes `dbcache/all_schemes.txt` behind a verify-everything gate).

```
# the record scheme, runnable + fuzzed vs the naive 27-mult loop
cargo test --release --lib mm55::   # 3 pass: fuzz(int), fuzz(2x2), 23*/55+-

cd matmul
# exact input-side minimizer: micro-optima + Sun's U/V optima + certs
python3 sidemin.py selftest
# exact output-side minimizer (transposition): constant + Sun 30 / cn122 28
python3 tcmin.py selftest

# the record, end to end: exact sides + transposition-exact output,
# emit the explicit 55-addition SLP, then re-verify it from scratch
python3 tcmin.py --emit /tmp/i19.slp external/i19-perminov56.bits
python3 verify_slp_file.py /tmp/i19.slp --trials 5000
#   integer trials PASS; non-commutative 2x2 PASS; 55 operations counted

# exact totals of the record chain (55 = 13+14+28 on cn122, ...)
python3 tcmin.py perminov_cache/bits/sun56.bits \
    external/i19-perminov56.bits external/i12-orbit56.bits \
    perminov_cache/bits/cr58-cn120.bits perminov_cache/bits/mws59.bits \
    external/stapleton60-orbitbest.bits

# class identity: cn122 is a de Groote class distinct from Sun's
python3 equiv.py classes perminov_cache/bits/sun56.bits \
    external/i19-perminov56.bits          # expect 2 (distinct classes)

# model-independent 54-exclusion: exact GF(2) output side via the
# transposition principle over GF(2), selftested against exhaustive
# search; with the GF(2) side tags it lower-bounds EVERY sign model
python3 gf2min.py selftest

# the database-wide closing sweep (54 excluded on all 17,376 classes):
# stage 1 census (seconds), then the 784-image orbit sweep per slot
python3 cfloor.py census          # writes dbcache/cfloor_cands.txt
python3 cfloor.py sweep i19w225c4efh-000 i46w213c23ci-016   # spot-check
#   full run: cfloor.py sweep-list dbcache/cfloor_cands.txt (~20 min)

# machine-checked proof (Lean 4 + Mathlib) that the 55-addition scheme
# equals A*B over a general non-commutative ring -- sorry-free
cd mm55proof && lake exe cache get && lake build
#   Matmul55.correct       depends on axioms: [propext]
#   Matmul55.scheme_eq_mul depends on axioms: [propext, Classical.choice, Quot.sound]
```

Deterministic caveats: kissat may return different sign models across versions (each returned model is $\mathbb{Z}$-verified), but the *committed representative* `external/i19-perminov56.bits` and the emitted SLP make the 55 deterministic to check.

8 Future work

Both sides are now minimized exactly, so the total for any given representative is its true additive complexity in this model. The open question is **whether 54 exists**:

- **More 55s (never fewer) inside the catalogue.** The 616 classes whose γ-orbit reaches output side 27 (§5) are the only places a further, fat-sided $55 = 28 + 27$ could live; pairing each with exact input sides at its cover-13 images is a bounded, well-defined hunt for record-*tying* schemes.
- **Beyond the published catalogue.** The optimality theorem is relative to the 17,376 published classes; our own 53 novel classes clear it with margin (every representative has output side ≥ 28 and input sides ≥ 28, so ≥ 56 total — they cannot even host a 55). A sub-55 scheme, if one exists, requires a rank-23 class nobody has catalogued — the de Groote classification at rank 23 is not known to be complete — or a cost model beyond ± 1 coefficients.
- **Independent certification.** The GF(2) floors rest on our minimizers (selftested against exhaustive search, quotient validated against the full orbit); DRAT-certified UNSAT instances of the same statements (companion benchmark paper) would make them solver-checkable end to end.
- **Database-wide exact re-score.** The input-side floors of all 17,376 classes are already charted (in the git-history census); pairing the low-floor classes with exact output sides is the systematic search for a second record.
- **Output-side lower bounds.** The exact output minimum is now known per representative via transposition; certifying it as a *class* optimum (that no representative does better) is a separate SAT/ILP problem — the subject of the companion benchmark note `doc/matmul_cxlb_satcomp.md`.
- The rank-22 question (challenge 4 of the HKS matrix-multiplication challenges, https://github.com/marijnheule/matrix-challenges) remains open and is where any result would change complexity theory rather than constants.

Acknowledgments

This work was carried out in an extended interactive collaboration with Claude (Anthropic; the Fable 5 and Opus 4.8 models), which implemented the repository's search tooling, exact minimizers, the Lean formalization, and much of this text under the author's direction and review. Every computational claim is mechanically checkable by the commands in the reproduction section; the results rest on those independent verifications rather than on trust in either the author or the tools.

References

Y. Sun. *An Exact 56-Addition, Rank-23 Scheme for General* 3×3 *Matrix Multiplication.* arXiv:2604.27645 (2026). Verifier and chain data cached at `matmul/perminov_cache/sun_verify.py`.

A. I. Perminov. *A 58-Addition, Rank-23 Scheme for General* 3×3 *Matrix Multiplication.* arXiv:2512.21980 (2025). Method paper: *Parallel Heuristic Exploration for Additive Complexity Reduction in Fast Matrix Multiplication.* arXiv:2512.13365 (2025); repo https://github.com/dronperminov/FastMatrixMultiplication.

E. Mårtensson, P. Stankovski Wagner, J. Stapleton. *A Rank 23 Algorithm for Multiplying* 3×3 *Matrices with an Arithmetic Complexity of 59.* arXiv:2601.05272 (2025).

J. Stapleton. *A 60-Addition, Rank-23 Scheme for Exact* 3×3 *Matrix Multiplication.* arXiv:2508.03857 (2025).

E. Mårtensson, P. Stankovski Wagner. *The Number of the Beast: Reducing Additions in Fast Matrix Multiplication Algorithms for Dimensions up to 666.* IACR ePrint 2024/2063 (Greedy-Potential; code `werekorren/fmm_add_reduction`).

O. Schwartz, N. Vaknin. *Pebbling Game and Alternative Basis for High Performance Matrix Multiplication.* SIAM J. Sci. Comput. 45(6), C614–C637 (2023).

P. Bürgisser, M. Clausen, M. A. Shokrollahi. *Algebraic Complexity Theory.* Grundlehren der math. Wissenschaften 315, Springer (1997) — the transposition principle / Tellegen's theorem (Ch. 13).

M. Heule, M. Kauers, J. Seidl. *Local Search for Fast Matrix Multiplication.* SAT 2019 (arXiv:1903.11391); *New ways to multiply* 3×3*-matrices.* J. Symb. Comput. 104 (2021), 899–916 (arXiv:1905.10192).

HKS scheme database (the 17,376-scheme corpus this note classifies against; source of the scheme identifiers): http://www.algebra.uni-linz.ac.at/research/matrix-multiplication/ (the `www.` host and plain http are required; https serves a self-signed cert). No separate GitHub mirror exists; the related GitHub artifact is the authors' SAT-benchmark repo https://github.com/marijnheule/matrix-challenges. Reproduction does not depend on the site: the corpus is the snapshot `schemes.tgz` (43,134,227 bytes, Last-Modified 2020-08-07, sha256 `4bc8132644504a917e3c076f64df8e6619fb67c55670179853ae5fdb1583074f`), archived at https://doi.org/10.5281/zenodo.21209925 (cite the HKS papers for the schemes, not the mirror).

J. Laderman. *A noncommutative algorithm for multiplying* 3×3 *matrices using 23 multiplications.* Bull. Amer. Math. Soc. 82(1), 126–128 (1976).

H. F. de Groote. *On varieties of optimal algorithms for the computation of bilinear mappings I–II.* Theor. Comput. Sci. 7 (1978), 1–24 and 127–148.

Appendix A: the 55-addition program (class `i19w225c4efh`)

The record scheme: exact input sides (13 + 14) and transposition-exact output side (28), 55 binary ± operations. On disk `matmul/external/i19-55adds-slp.txt`; bits at `matmul/external/i19-perminov56.bits` (Perminov's published cn122 scheme, our sign model m2). Independently re-verified by `matmul/verify_slp_file.py` (5,000 integer + 300 non-commutative 2×2-block trials; 55 operations counted), and transcribed to Rust in `src/mm55.rs`.

```
# 3x3x3 r=23: 13(A) + 14(B) + 28(C) = 55 additions; M_i = P_i * Q_i
## A-side
aw0 = a13 - a23
aw1 = a11 - aw0
aw2 = a12 + aw1
aw3 = -a23 + aw2
aw4 = -a22 + aw2
aw5 = a33 + aw3
aw6 = -a12 + aw5
aw7 = -a21 + aw4
aw8 = a31 + aw7
aw9 = a32 + aw8
aw10 = -a32 + aw5
aw11 = a21 - a31
aw12 = aw1 - aw11
P1 = aw2
P2 = aw8
P3 = a11
P4 = a32
P5 = a11
P6 = aw6
P7 = aw10
P8 = a33
P9 = aw9
P10 = aw1
P11 = a33
P12 = aw3
P13 = a23
P14 = aw7
P15 = a21
P16 = a31
P17 = a31
P18 = aw0
P19 = aw5
P20 = aw12
P21 = a12
```

```
P22 = a22
P23 = aw4
## B-side
bw0 = b13 + b33
bw1 = b11 + b31
bw2 = b11 - b21
bw3 = b11 + b13
bw4 = b23 + b33
bw5 = b13 + bw2
bw6 = bw2 - bw4
bw7 = -b23 + bw5
bw8 = b32 + bw1
bw9 = b12 + bw8
bw10 = -bw6 + bw8
bw11 = b22 + bw10
bw12 = -b21 + bw10
bw13 = -b23 + bw12
Q1 = bw10
Q2 = bw5
Q3 = bw0
Q4 = b22
Q5 = bw9
Q6 = bw4
Q7 = b23
Q8 = b31
Q9 = b21
Q10 = bw6
Q11 = b32
Q12 = bw13
Q13 = b32
Q14 = bw3
Q15 = b12
Q16 = b12
Q17 = b13
Q18 = bw1
Q19 = bw12
Q20 = bw2
Q21 = bw11
Q22 = b22
Q23 = bw7
## products
M1 = P1 * Q1
M2 = P2 * Q2
M3 = P3 * Q3
M4 = P4 * Q4
M5 = P5 * Q5
M6 = P6 * Q6
M7 = P7 * Q7
M8 = P8 * Q8
M9 = P9 * Q9
M10 = P10 * Q10
M11 = P11 * Q11
M12 = P12 * Q12
M13 = P13 * Q13
M14 = P14 * Q14
M15 = P15 * Q15
M16 = P16 * Q16
M17 = P17 * Q17
M18 = P18 * Q18
M19 = P19 * Q19
M20 = P20 * Q20
M21 = P21 * Q21
M22 = P22 * Q22
M23 = P23 * Q23
## C-side
cw0 = -M19 + M6
cw1 = cw0 + M11
cw2 = M8 + cw1
cw3 = -M12 - cw2
```

```
cw4 = -M17 - cw3
cw5 = M2 + cw4
cw6 = M1 + -M13
cw7 = cw6 + M10
cw8 = -M14 + -M12
cw9 = cw8 + cw5
cw10 = M18 + cw7
cw11 = cw2 + cw10
cw12 = M5 + M21
cw13 = cw12 - cw10
cw14 = M3 + cw3
cw15 = cw7 + -M20
cw16 = cw15 + cw9
cw17 = M15 + M22
cw18 = cw17 - -M13
cw19 = M23 - cw5
cw20 = cw19 - M10
cw21 = cw20 - -M20
cw22 = M9 - cw1
cw23 = cw22 + cw9
cw24 = M4 + M16
cw25 = cw24 + M11
cw26 = -M7 + M6
cw27 = cw26 - cw4
C11 = cw11
C12 = cw13
C13 = cw14
C21 = cw16
C22 = cw18
C23 = cw21
C31 = cw23
C32 = cw25
C33 = cw27
```

Appendix B: the side-minimizer on a worked example

Rows (targets) over base variables a, b, c, d: $T_1 = a + b$ and $T_2 = a + b + c + d$. Both have ≥ 2 terms, are distinct up to global sign, and neither is a basis vector, so nt $= 2$ and any chain needs at least 2 additions.

Slack $h = 0$ (no helpers — every addition must create a target): the pool starts as $\{a, b, c, d\}$. $T_1 = a + b$ is derivable (both operands in the pool): cover it; the pool grows to $\{a, b, c, d, a{+}b\}$. Now T_2 must be $x \pm y$ with x, y in the pool: subtracting each pool value from T_2 gives $c{+}d$, $a{+}c{+}d$, $b{+}c{+}d$, $a{+}b{+}c$, $a{+}b{+}d$ — none in the pool, so the closure is at a fixpoint with T_2 uncovered, and $h = 0$ is **exhausted: 2 additions are impossible.**

Slack $h = 1$: at that same fixpoint the *enabling set* — values u that would directly complete an uncovered target as $T_2 = x \pm u$ — is $\{c{+}d,\ a{+}c{+}d,\ b{+}c{+}d,\ a{+}b{+}c,\ a{+}b{+}d, \dots\}$; of these, $u = c + d$ is itself derivable from the pool (c and d are basis vectors). Insert it as the helper, and the closure resumes: $T_2 = (a{+}b) + (c{+}d)$.

The optimum is nt $+ h^* = 2 + 1 =$ **3 additions**, found in 3 search nodes; the emitted chain is

```
w0 = a + b        (covers T1)
w1 = c + d        (helper)
w2 = w0 + w1      (covers T2)
```

replay-verified by expansion. Global signs are free throughout: a row $-a - b$ is the same target as $a + b$ (a chain value may be used negated at no cost).

Appendix C: exact scoring end to end (sun56)

The pipeline of §3–§4 on Sun's representative, every number reproduced by `sidemin.py --models 24 perminov_cache/bits/sun56.bits` and `tcmin.py perminov_cache/bits/sun56.bits`:

1. **Mod-2 gate.** The 621-bit vector satisfies all 729 Brent equations over GF(2).

2. **Sign-SAT.** The support has 175 nonzero coefficients (175 sign variables); the 729 integer Brent equations have 453 covering terms in total, and the instance has 4,269 clauses. Two concrete equations: one type-3 equation is covered by $k = 3$ terms (products M_2, M_{17}, M_{23}) with right-hand side 1, so **exactly** $(3-1)/2 = 1$ **of the three terms must be negative**; a neighboring rhs-0 equation is covered by $k = 4$ terms (M_2, M_{17}, M_{20}, M_{23}), so exactly 2 of 4 are negative. kissat solves the instance in milliseconds; each returned model is $\mathbb{Z}$-verified before use.

3. **Exact input sides** (model m0). The 23 A-rows contain 12 distinct multi-term targets; $h = 0$ is exhausted and $h^* = 1$ succeeds: A-side $= 12 + 1 =$ **13** (3 search nodes). The B-rows contain 11 distinct targets; $h = 0$ and $h = 1$ are exhausted, $h^* = 2$ succeeds: B-side = **13** (11 nodes). These reproduce Sun's own input-side optima and impossibility certificates.

4. **Exact output side** (transposition). The transpose W^{T} (23 rows over the 9 outputs) has additive cost 16 by `sidemin`, so the output side is $16+14 =$ **30** — equal to greedy here, confirming Sun's representative is output-optimal.

5. **Total.** $13 + 13 + 30 =$ **56**, the true additive complexity of Sun's representative.